
Performance Incident and Root-Cause Knowledge Base

Diagnostic patterns, competing hypotheses, evidence checks, remediations, and validation steps

Corpus ID	PTA-RCA-KB-V2
Version	2.0 - Expanded RAG Edition
Source dataset	performance_test_dataset_1500_unique_names.csv
Rows represented	1,500
Primary consumer	Analysis Agent inside the Performance Test Agent

Document purpose

- Give the Analysis Agent reusable diagnostic knowledge beyond individual historical records.
- Describe each bottleneck using symptoms, confirming evidence, counter-evidence, root cause, remediation, and validation.
- Reduce hallucination by teaching the agent when not to apply a familiar recommendation.
- Map database, CPU, memory, thread, network, HTTP, and mixed-signal incidents to explainable conclusions.

Data handling note: This document uses aggregate metrics and selected Job IDs. Customer names and account values are intentionally excluded because they do not improve performance root-cause analysis. All content is synthetic and intended for RAG development, demonstrations, and testing.

Recommended RAG metadata

Field	Example value
document_type	Performance Incident and Root-Cause Knowledge Base
domain	banking-performance-testing
agent	analysis-agent
confidentiality	dummy-synthetic
chunk_strategy	section-aware, 500-900 tokens, 100-token overlap
retrieval_keys	scenario, API, status, symptom, bottleneck, recommendation, SLA

1. Diagnostic taxonomy and evidence model

A root cause is a hypothesis supported by multiple independent signals.

The dataset uses five bottleneck labels: High CPU, Memory Pressure, Database Pool, Thread Contention, and Network Latency. These labels are useful starting points, but a production Analysis Agent should evaluate competing causes before issuing a recommendation.

Layer	Typical symptoms	Primary evidence	Common confounder
Application compute	High CPU, TPS plateau, rising latency	CPU per instance, profiler, method timing	Database waits can also consume threads
Application memory	High heap, long pauses, allocation spikes	Heap usage, GC pause, allocation rate	OS page cache or container limit
Concurrency	Queue growth, rejected work, lock waits	Thread pool active count, queue depth, thread dump	Slow downstream dependency
Database	Connection saturation, slow SQL, lock waits	Pool active/wait, query latency, DB locks	Too-small pool versus connection leak
Network/dependency	502/504, normal local resources, variable latency	Gateway timing, dependency spans, packet loss	Application timeout setting
Security controls	401/403/429, token or rate-limit behavior	Auth logs, policy decisions, gateway counters	Functional error mislabeled as performance

Evidence hierarchy

- Level 1 - Observation: measured values such as response time, CPU, memory, pool usage, status code, and alert timestamps.
- Level 2 - Correlation: two or more signals change together, for example response time rises as pool wait time rises.
- Level 3 - Mechanism: a technical explanation connects the signals, such as requests waiting because all database connections are active.
- Level 4 - Validation: a controlled fix changes the expected metric and restores the SLA.
- The Analysis Agent should reserve high confidence for Level 3 or Level 4 evidence.

2. Database connection pool incidents

Knowledge entries for saturation, leaks, slow queries, and lock amplification.

DB-001: Pool capacity exhausted

Symptoms: DB connections near configured maximum; response time rises; requests wait; 500/503/504 possible

Confirming evidence: Pool active count, pending borrowers, acquisition timeout

Recommended actions: Increase pool only after database capacity review; reduce query duration; add back-pressure

Validation: Latency and acquisition waits fall without database overload

DB-002: Connection leak

Symptoms: Active connections rise over time and do not return after load decreases

Confirming evidence: Leak detection logs, long-held connection stack traces

Recommended actions: Close connections in finally/context manager; fix transaction boundaries

Validation: Active count returns to baseline after requests complete

DB-003: Slow query amplification

Symptoms: Pool appears full because each query holds a connection too long

Confirming evidence: Top SQL by total time, execution plan, table scans

Recommended actions: Add index, rewrite query, reduce returned columns, update statistics

Validation: Query duration and pool occupancy both decline

DB-004: Database lock contention

Symptoms: Connections are active but blocked; latency has step-like spikes

Confirming evidence: Lock wait graph, blocked session tree, transaction duration

Recommended actions: Shorten transactions; change lock order; add index; reduce hot-row contention

Validation: Lock wait time and timeout count decrease

DB-005: Pool oversized for database

Symptoms: Application has many connections but DB CPU or I/O collapses

Confirming evidence: DB CPU, context switching, max connections, throughput after pool changes

Recommended actions: Reduce pool per instance; cap concurrency; scale database appropriately

Validation: Throughput improves with fewer concurrent DB sessions

DB-006: Replica or dependency lag

Symptoms: Read queries are slow even when pool is not exhausted

Confirming evidence: Replica lag, network timing, read routing logs

Recommended actions: Repair replication, route critical reads, adjust consistency strategy

Validation: Lag and read latency normalize

Do not diagnose database pool exhaustion solely because DB Connections is numerically high. Compare the value with the configured maximum and inspect wait time. A large pool with many idle connections is not the same as saturation.

3. High CPU incidents

Different causes can produce the same high-CPU metric and require different fixes.

CPU-001: Insufficient compute capacity

Symptoms: CPU > 90% across instances, TPS stops scaling, no single hot method

Confirming evidence: Per-instance CPU, load balance, run queue, throughput curve

Recommended actions: Scale horizontally or vertically after confirming even load distribution

Validation: TPS increases and latency decreases at same workload

CPU-002: Hot code path

Symptoms: One method or stack consumes disproportionate CPU

Confirming evidence: Continuous profiler, flame graph, method timing

Recommended actions: Optimize algorithm, reduce serialization, cache repeated computation

Validation: Hot method CPU share falls

CPU-003: Encryption or signature overhead

Symptoms: CPU increases with request volume on secure payment operations

Confirming evidence: Crypto method profile, key operation timing

Recommended actions: Use efficient libraries, hardware acceleration, connection reuse

Validation: Crypto CPU per request decreases without weakening security

CPU-004: Busy retry loop

Symptoms: CPU high while downstream dependency is failing

Confirming evidence: Retry counters, logs, stack samples

Recommended actions: Add bounded exponential backoff and circuit breaker

Validation: Retry rate and CPU drop during dependency failure

CPU-005: Excessive logging

Symptoms: CPU and I/O increase with verbose synchronous logs

Confirming evidence: Log volume per request, formatter profile, disk latency

Recommended actions: Reduce log level, async logging, structured compact events

Validation: Log overhead and latency decline

CPU-006: Uneven load distribution

Symptoms: One instance saturates while peers remain underused

Confirming evidence: Per-pod CPU, request count, sticky session behavior

Recommended actions: Fix load balancing, rebalance partitions, review affinity

Validation: CPU becomes balanced across nodes

4. Memory pressure and garbage-collection incidents

Heap, native memory, cache, and container limits must be separated.

MEM-001: Heap too small for steady workload

Symptoms: Heap repeatedly approaches maximum; frequent collections; latency oscillates

Confirming evidence: Heap after GC, GC frequency, pause time

Recommended actions: Right-size heap; reduce live set; tune collector

Validation: Stable post-GC heap and shorter pauses

MEM-002: Memory leak

Symptoms: Post-GC heap trends upward across the test

Confirming evidence: Heap dump dominator tree, object retention paths

Recommended actions: Fix retained references, listener cleanup, cache bounds

Validation: Post-GC baseline no longer climbs

MEM-003: Unbounded cache

Symptoms: Memory grows with key cardinality and hit ratio may be low

Confirming evidence: Cache entry count, eviction count, key distribution

Recommended actions: Set maximum size and TTL; prevent high-cardinality keys

Validation: Cache size stabilizes with acceptable hit rate

MEM-004: Large payload retention

Symptoms: Large request/response objects remain live during processing

Confirming evidence: Allocation profile, payload size, buffering behavior

Recommended actions: Stream data, limit payload, avoid duplicate buffers

Validation: Peak memory per request decreases

MEM-005: Native or direct-buffer pressure

Symptoms: Container memory rises while Java heap looks normal

Confirming evidence: Native memory tracking, direct buffer count

Recommended actions: Bound direct buffers, update library, tune network buffers

Validation: RSS and native allocations stabilize

MEM-006: Container OOM or throttling

Symptoms: Process restarts or 502/503 during high memory

Confirming evidence: Container events, OOMKilled, cgroup limit

Recommended actions: Set realistic limits and requests; reduce footprint

Validation: No OOM events at target workload

5. Thread contention, queues, and worker exhaustion

Concurrency failures may occur with either high or moderate CPU.

THR-001: Worker pool exhausted

Symptoms: All workers active, queue grows, latency rises

Confirming evidence: Active workers, queue depth, rejection count

Recommended actions: Tune pool from measured service time; remove blocking calls; add bulkheads

Validation: Queue remains bounded and rejection rate falls

THR-002: Lock contention

Symptoms: Many threads BLOCKED or waiting on same monitor

Confirming evidence: Thread dump, lock owner, wait duration

Recommended actions: Reduce critical section, partition shared state, use suitable concurrency primitive

Validation: Blocked-thread count falls

THR-003: Downstream blocking

Symptoms: Workers wait on database or external API

Confirming evidence: Distributed traces, socket waits, dependency latency

Recommended actions: Set timeouts, use async I/O where appropriate, isolate dependency pool

Validation: Worker utilization falls while throughput improves

THR-004: Thread creation storm

Symptoms: Thread count grows rapidly; context switching rises

Confirming evidence: Thread count trend, creation stack, OS context switches

Recommended actions: Use bounded executors; reuse clients; stop per-request thread creation

Validation: Thread count remains stable

THR-005: Queue too large

Symptoms: Errors are delayed but tail latency becomes extreme

Confirming evidence: Queue residence time, queue length, p99 latency

Recommended actions: Use bounded queue and back-pressure; reject early when necessary

Validation: Tail latency is controlled and failures are explicit

THR-006: Deadlock

Symptoms: Throughput reaches zero for affected path

Confirming evidence: Deadlock detector, thread dump cycle

Recommended actions: Fix lock ordering and timeout strategy

Validation: No deadlock under repeated stress test

6. Network latency, gateway, and dependency incidents

Use timing decomposition to avoid blaming the application without evidence.

NET-001: Gateway timeout

Symptoms: HTTP 504 near configured timeout; back end may finish later

Confirming evidence: Gateway access log, upstream time, application completion time

Recommended actions: Fix slow upstream; align timeout budgets; do not only raise timeout

Validation: Requests complete within budget and 504 count falls

NET-002: Bad gateway or reset

Symptoms: HTTP 502, connection reset, pod restart, or invalid upstream response

Confirming evidence: Gateway error reason, upstream reset, instance health

Recommended actions: Stabilize upstream, health checks, graceful shutdown

Validation: No resets during deployment and load

NET-003: Connection establishment overhead

Symptoms: Latency includes DNS, TCP, or TLS setup per request

Confirming evidence: Client timing phases, connection reuse rate

Recommended actions: Enable keep-alive and pooling; optimize DNS/TLS

Validation: Connection setup time and socket count decline

NET-004: Packet loss or unstable path

Symptoms: Variable latency and retransmissions with normal app resources

Confirming evidence: Network telemetry, retransmit rate, path comparison

Recommended actions: Repair route, network interface, or service mesh issue

Validation: Retransmits and latency variance decrease

NET-005: Rate limiting

Symptoms: HTTP 429 or gateway rejection at a stable threshold

Confirming evidence: Rate-limit counters, policy logs

Recommended actions: Adjust approved limits; smooth load; client backoff

Validation: Accepted throughput follows approved capacity

NET-006: Slow downstream service

Symptoms: Local CPU/memory normal; trace span dominated by dependency

Confirming evidence: Distributed trace and dependency SLO

Recommended actions: Optimize or scale dependency; apply circuit breaker/cache

Validation: Dependency span meets its allocated budget

7. HTTP status interpretation and counterexamples

Status codes guide investigation but do not uniquely identify the cause.

Status	Performance meaning	Possible causes	Required checks
200	Request completed, but may still violate latency SLA	Slow success, dependency delay, overloaded system	Compare response time and percentile SLA
500	Application-level failure	Unhandled exception, thread issue, DB error, code defect	Exception signature, stack trace, correlated resource signal
502	Gateway received invalid or reset upstream response	Instance restart, connection reset, proxy/upstream mismatch	Gateway reason, pod lifecycle, upstream logs
503	Service unavailable or overloaded	No healthy instance, rejection, maintenance, circuit breaker	Health checks, queue/rejection, deployment state
504	Gateway timeout waiting for upstream	Slow app, database wait, dependency, network	Timeout budget and distributed timing
401/403	Authentication or authorization result, usually functional/security	Expired token, policy denial, clock skew	Security Agent output; token and policy logs
429	Rate limit or quota	Gateway policy, protective throttling, client burst	Rate-limit counters and approved capacity

Counterexample rules

- HTTP 504 plus high DB connections suggests database wait, but HTTP 504 plus normal resources suggests network or downstream delay.
 - HTTP 503 plus high CPU may indicate overload, but HTTP 503 during a deployment may indicate missing healthy instances.
 - HTTP 500 with low response time is more likely a fast functional exception than a performance bottleneck.
 - HTTP 200 with response time above SLA is still a performance failure and should be analyzed.
-

8. Multi-signal diagnosis and confidence scoring

A practical framework for converting retrieved evidence into an explainable conclusion.

Rule	Condition	Primary hypothesis	Confidence modifier
MS-01	CPU >= 90 and DB < 300 and memory < 85	High CPU / compute saturation	+20 if profiler confirms hot path
MS-02	DB >= 400 and response >= 2000	Database pool or slow-query pressure	+20 if pool wait > 0
MS-03	Memory >= 88 and response >= 2000	Memory pressure / GC	+20 if GC pause correlates
MS-04	HTTP 504 and all local resources moderate	Network or downstream timeout	+15 if dependency span dominates
MS-05	High latency and worker queue rising	Thread pool exhaustion or blocking	+20 if thread dump confirms waiting
MS-06	Only one weak signal and no historical match	Unknown / insufficient evidence	Cap confidence at 55%
MS-07	Retrieved cases disagree on root cause	Present top two hypotheses	Reduce confidence by 15
MS-08	Controlled remediation restores SLA	Validated root cause	Confidence 90-99%

Recommended response structure

```
{
  "status": "FAIL",
  "probable_root_causes": [
    {"cause": "Database Pool", "confidence": 0.82, "evidence": ["db_connections=470", "response_ms=3200", "similar_job=JOB-..."]}
  ],
  "counter_evidence": ["cpu_not_saturated"],
  "recommended_checks": ["pool wait time", "slow SQL", "connection leak"],
  "recommended_actions": ["optimize query", "review pool capacity"],
  "citations": ["DB-001", "DB-003", "historical incident ID"]
}
```

9. Remediation validation and anti-patterns

A recommendation is incomplete until its effect is measured.

Change	Expected improvement	Risk / anti-pattern	Validation metric
Increase DB pool	Lower acquisition wait if DB has spare capacity	Oversizing can overload DB	Pool wait, DB CPU, TPS, p95
Increase heap	Fewer collections if live set fits	Can create longer pauses and hide leak	GC pause, post-GC heap, RSS
Increase thread pool	More concurrency if work is CPU-light and downstream can handle it	Can increase contention and DB pressure	Queue time, CPU, dependency saturation
Scale application	More capacity for parallelizable CPU work	Does not fix shared DB or lock bottleneck	Per-instance CPU and total TPS
Raise timeout	May reduce timeout errors temporarily	Can increase queues and user wait	Completion rate plus tail latency
Add cache	Can reduce DB/dependency calls	Stale data or unbounded memory	Hit rate, DB load, memory
Retry failed calls	Can recover transient errors	Retry storm amplifies outage	Retry rate, dependency load, success gain

RCA quality checklist

- State the observed evidence before stating the cause.
- Include at least one competing hypothesis when evidence is incomplete.
- Explain why the selected cause fits better than the alternatives.
- Recommend diagnostic checks before risky configuration changes.
- Describe how to validate the fix under the same workload profile.
- Cite the retrieved knowledge entry and historical execution used.

End of incident knowledge base. Combine these diagnostic patterns with real monitoring metrics, logs, traces, database telemetry, and the structured dataset.